



DALHOUSIE UNIVERSITY

Advanced Topic in Cloud Computing

CSCI 5409

Final Term Project

Report by – Akanksha Raut

Banner ID – B01055690

Table of Contents

1. Project Introduction	3
1.1 Overview.....	3
1.2 Functional Objective	3
1.3 Target Users	3
1.4 Performance Targets.....	3
2. AWS Service Category Requirements & Comparison.....	4
3. Final Architecture	5
3.1 Architecture Overview	5
3.2 High-Level Data Flow.....	5
3.2 Component Interaction	6
3.3 Programming Languages & Tooling.....	6
3.4 Deployment Workflow	6
3.4 Deployment Workflow	6
4. Security Architecture.....	7
4.1 Network Layer Security.....	7
4.2 API Layer Security	7
4.3 Storage Layer Security	7
4.4 Access Control (IAM)	7
4.5 Audit & Monitoring	7
4.6 Strengths and Vulnerabilities.....	7
Security Mechanisms used –	8
5. Cost Analysis (CAD).....	8
5.1 Baseline Monthly Cost (100 Users/Month)	8
5.2 Scaling Cost (10,000 Users/Month)	9
5.3 Higher Cost Enhancements	9
6. Implementation Screenshots	10
7. Future Improvement	11
8. Conclusion	11

PathMentor - A Context-Aware Mentorship Platform

1. Project Introduction

1.1 Overview

PathMentor is a context-aware mentorship and differentiation platform designed to bridge the gap between academic theory and practical community experiences. Unlike traditional static mentorship programs, PathMentor leverages a decentralized database of real-world "mentorship footprints" and Amazon Bedrock AI to generate situational guidance for individuals seeking to differentiate themselves in competitive fields.

1.2 Functional Objective

The core objective of PathMentor is to provide users with a **differentiation roadmap**. By analyzing historical community experiences, such as specific internship routines, unique project pivots, or interview strategies the platform calculates what a user should do *differently* compared to the average path, based on their exact user type and professional goals.

1.3 Target Users

PathMentor is designed for three primary categories of users:

- **CS Students:** Individuals seeking co-ops or initial internships who need help identifying non-obvious ways to stand out.
- **New Graduates:** Professionals early in their careers aiming for their first full-time role or looking to switch domains.
- **Professionals:** Experienced individuals seeking promotions or strategic skill improvement through specific, high-impact community insights.

1.4 Performance Targets

To ensure a high-quality user experience, PathMentor adheres to the following technical benchmarks:

- **Low Latency:** API response times for AI-generated roadmaps must remain under 30 secs for 95% of standard requests.
- **High Availability:** The architecture aims for 99.9% uptime, leveraging AWS regional serverless services to minimize downtime.
- **On-Demand Scalability:** The system must automatically handle bursts of traffic (e.g., during internship application seasons) without manual intervention or performance degradation.

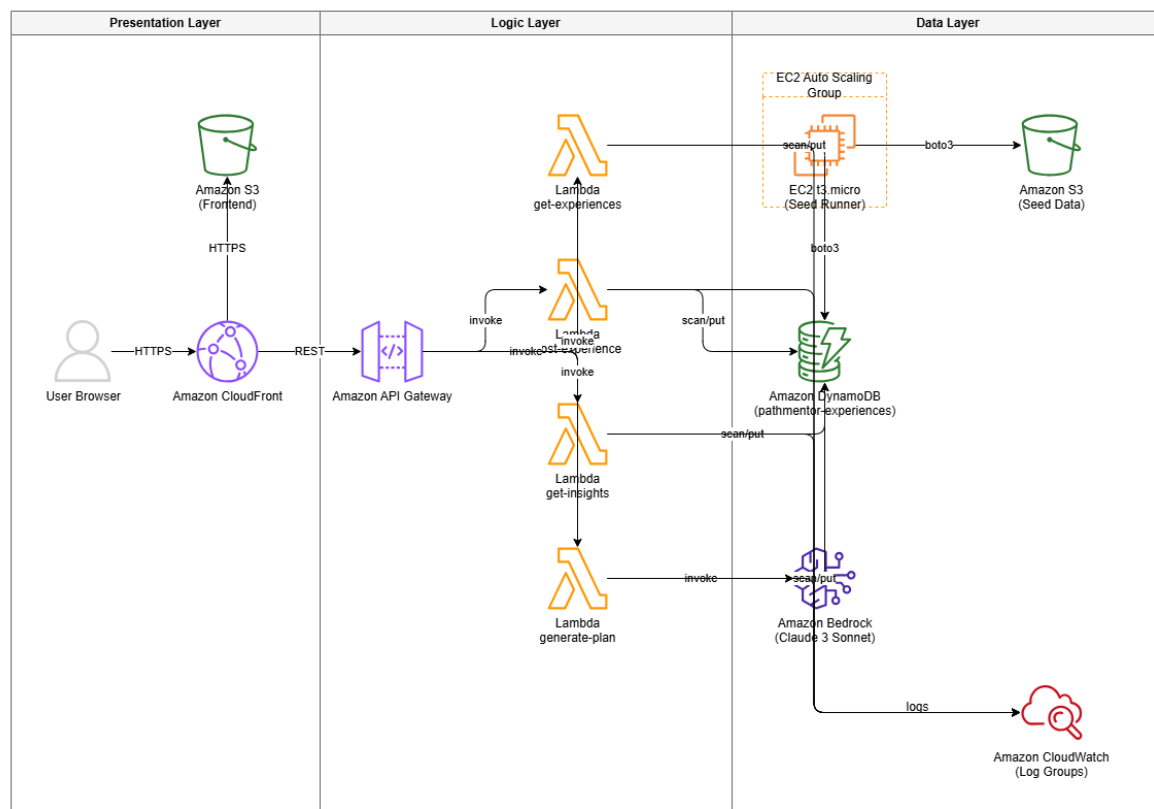
2. AWS Service Category Requirements & Comparison

Category	Selected Service	Alternative Considered	Selection Rationale
Compute #1	AWS Lambda	Amazon EC2	PathMentor uses 4 microservices (experiences, insights, plan). Lambda's event-driven model matches our API traffic, avoiding the \$14+/month cost of idle EC2 instances.
Compute #2	EC2 + ASG	AWS Fargate	Used specifically for the Seed Runner. Since seeding is a one-time intensive operation at launch, an EC2 t3.micro instance is easier to manage via User Data scripts in an AWS Academy environment than containerizing for Fargate.
Database	Amazon DynamoDB	Amazon RDS	Our data (experiences) is semi-structured. DynamoDB's NoSQL model allowed us to implement a Global Secondary Index (GSI) for role-based filtering (user_type-goal-index) without the schema rigidity of SQL.
Storage	Amazon S3	Amazon EFS	S3 is used for both frontend hosting and holding the seed_data.py scripts. Its serverless nature and low cost (\$0.023/GB) make it superior to EFS for static assets.
Networking	API Gateway	ALB	REST API Gateway integrates natively with Lambda and provides built-in throttling and CORS management, which was critical for our Vite-based frontend.
CDN	CloudFront	S3-only hosting	CloudFront enables global lower latency and provides TLS 1.2/1.3 encryption for the frontend, which S3 static websites cannot provide natively with custom domains.
Ai/ML	Amazon Bedrock	Groq api	Powers the <i>Generative Plan</i> engine using Claude 3 models (Haiku for the project) to synthesize personalized mentorship roadmaps from raw community data.

Iac	Terraform	AWS CloudFormation	All infrastructure is defined as HCL (HashiCorp Configuration Language), ensuring reproducibility across environments.
Management	Amazon CloudWatch	Datadog	Native logging/monitoring; chosen for deep integration and native Lambda log integration with no additional SDK required

3. Final Architecture

3.1 Architecture Overview



3.2 High-Level Data Flow

In summary, the platform's data flow occurs across two primary channels:

- **Interactive Flow:** 'User Client > CloudFront > Amazon S3 (Static Website) '
- **Application Flow:** 'User Client > API Gateway > Private Subnet (Lambda Services) > DynamoDB / Bedrock AI'

1. **Presentation Layer:** A React-based Single Page Application (SPA) hosted in Amazon S3 and distributed via Amazon CloudFront.
2. **Logic Layer:** A suite of Python-based AWS Lambda functions triggered by an Amazon API Gateway REST endpoint.
3. **Data Layer:** Amazon DynamoDB stores community experiences, while Amazon Bedrock processes these inputs to generate AI insights.

3.2 Component Interaction

- **Step 1:** The user interacts with the React frontend. Static assets are retrieved from **CloudFront Edge Locations** (caching).
- **Step 2:** The frontend makes a RESTful call to **API Gateway**.
- **Step 3:** API Gateway triggers the appropriate **AWS Lambda** function (e.g., 'generate_plan' or 'get_experiences').
- **Step 4:** The Lambda function queries **DynamoDB** for community data.
- **Step 5:** For roadmap generation, the Lambda function sends the data to **Amazon Bedrock (Claude 3 Haiku)** for analysis.
- **Step 6:** The structured response is returned through API Gateway to the frontend for display.

3.3 Programming Languages & Tooling

- **Python (3.11):** Used for all backend Lambda logic due to its robust ecosystem for data manipulation and AWS SDK (Boto3) integration.
- **Javascript/React:** Used for the frontend application to provide a dynamic, reactive user interface.
- **HCL (HashiCorp Language):** Used for Terraform infrastructure-as-code to manage the AWS resource lifecycle.
- **Bash Shell:** Used locally for deployment scripts and automated frontend builds.

3.4 Deployment Workflow

Deployment is fully automated through two main steps:

1. **Infrastructure Provisioning:** 'terraform apply' creates the entire AWS stack in a consistent state.
2. **Application Deployment:** A customized 'deploy.sh' script builds the React assets, updates the frontend environment variables (API URLs), and synchronizes the distribution to S3 and CloudFront.

3.4 Deployment Workflow

Deployment is fully automated through two main steps:

1. **Infrastructure Provisioning:** 'terraform apply' creates the entire AWS stack in a consistent state.

2. **Application Deployment:** A customized 'deploy.sh' script builds the React assets, updates the frontend environment variables (API URLs), and synchronizes the distribution to S3 and CloudFront.

4. Security Architecture

4.1 Network Layer Security

PathMentor uses **Amazon CloudFront** to enforce ****SSL/TLS (HTTPS)**** for all frontend traffic. By using ****Regional API Gateway**** endpoints, the platform ensures that API calls are localized to the deployment region, minimizing external exposure.

4.2 API Layer Security

Lambda functions return Access-Control-Allow-Origin, which allows all origins, it's a known vulnerability to address in future work.

4.3 Storage Layer Security

The frontend S3 buckets are configured with ****Public Access Block****. All access is restricted to CloudFront through ****Origin Access Control (OAC)****, ensuring that the source files cannot be accessed directly via the internet.

4.4 Access Control (IAM)

The project utilizes the **Principle of Least Privilege**:

- **Execution Roles:** Lambda functions are assigned specific IAM roles with "Read" or "Write" permissions limited only to the required DynamoDB table.
- **Boundary Control:** The AWS Academy **LabRole** is used to ensure all operations remain within the academic sandbox environment.

4.5 Audit & Monitoring

- **Amazon CloudWatch** is configured to capture:
- **Execution Logs:** All Lambda errors and console logs.
- **API Metrics:** Count of 4xx and 5xx errors.
- **System Health:** EC2 CPU usage and DynamoDB throttled requests.

4.6 Strengths and Vulnerabilities

- **Strengths:** High isolation of components; serverless scaling reduces the surface area for traditional OS-level attacks.
- **Remaining Vulnerability:** The current version lacks session-based user authentication (e.g., AWS Cognito).

- **Planned Improvement:** Integration of Amazon Cognito for JWT-based authentication and user-specific data isolation.

Security Mechanisms used –

Mechanism	Choice	Rationale
Encryption	AES-256	Default for KMS-managed encryption in DynamoDB.
Access Control	IAM Roles (LabRole)	Adheres to the Principle of Least Privilege. Only specific Lambda functions can access the <code>InvokeModel</code> action for Bedrock.
Vulnerability Fix	API Throttling	Configured in API Gateway to prevent DDoS attacks from consuming the Bedrock token budget.

5. Cost Analysis (CAD)

**Estimates based on current AWS pricing for the Canada (Central)*

5.1 Baseline Monthly Cost (100 Users/Month)

For a small-scale deployment of 100 users per month, the platform operates almost entirely within the **AWS Free Tier**.

AWS Service	Usage Estimate	Unit Price (CAD)	Monthly Cost (CAD)
AWS Lambda	~50,000 invocations, 256MB, avg 2s	Free Tier (1M req free)	\$0.00
Amazon DynamoDB	~10,000 reads, ~2,000 writes	Free Tier (25 WCU/RCU free)	\$0.00
Amazon S3 (frontend + seed)	~500MB storage, ~5GB transfer	Free Tier (5GB free)	\$0.00
Amazon CloudFront	~10GB data transfer	Free Tier (1TB/month free)	\$0.00
Amazon API Gateway	~50,000 REST API calls	Free Tier (1M calls/month free)	\$0.00
Amazon Bedrock (Claude 3 Haiku)	~100 roadmap generations, ~2,000 tokens avg	\$0.005 CAD/1K input tokens	~\$1.50
EC2 t3.micro (seed runner)	1 instance, ~2 hrs total runtime	Free Tier (750 hrs/month free)	\$0.00
Amazon CloudWatch	Lambda log ingestion ~100MB	Free Tier (5GB ingestion free)	\$0.00
Total			~\$1.50 CAD

5.2 Scaling Cost (10,000 Users/Month)

Scaling to 10,000 users increases usage-based costs but remains highly affordable due to the serverless model.

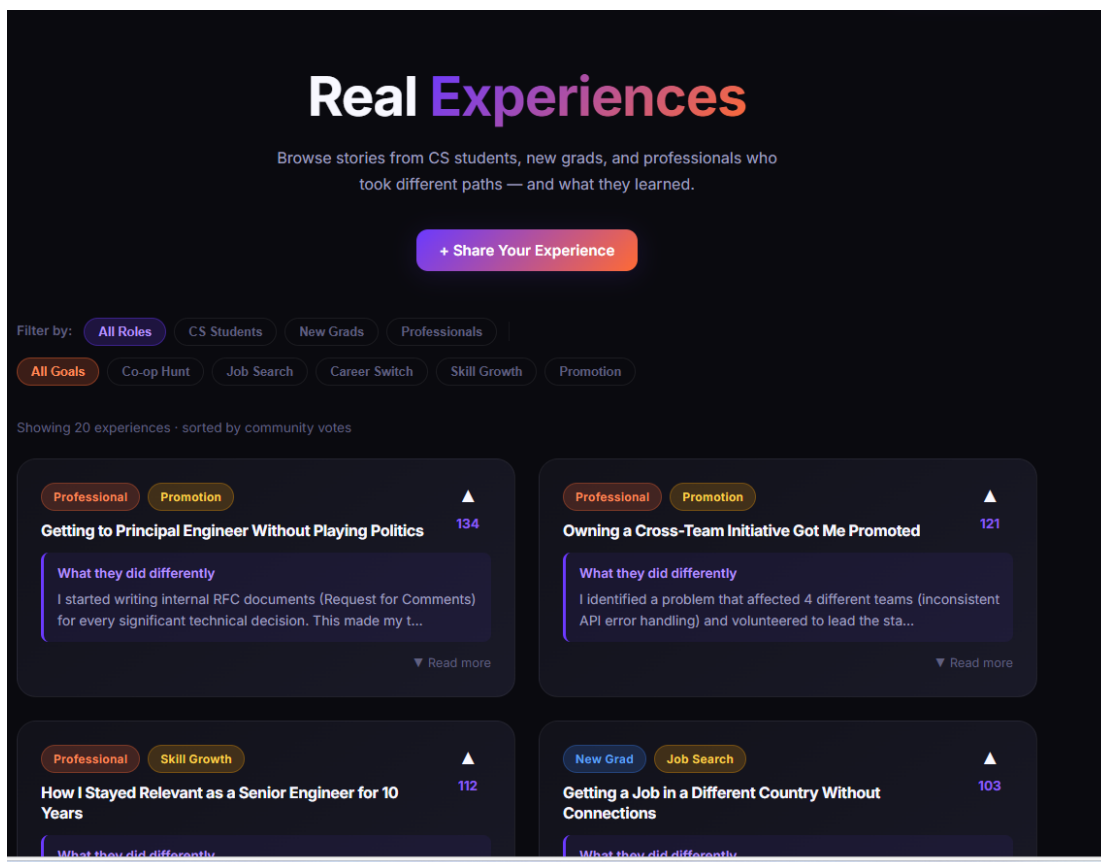
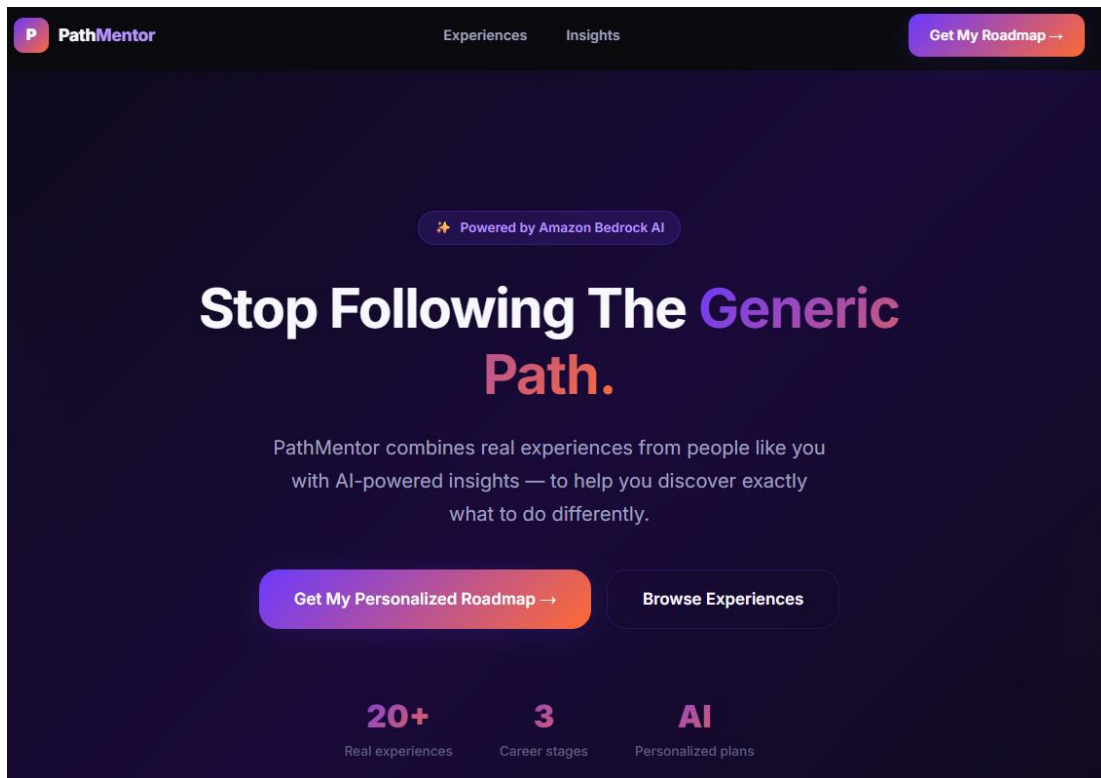
AWS Service	Usage Estimate	Unit Price (CAD)	Monthly Cost (CAD)
AWS Lambda	~5M invocations, 256MB, avg 2s	\$0.00002778 CAD/GB-sec	~\$4.50
Amazon DynamoDB	~1M reads, ~200K writes	\$0.169 CAD/million reads	~\$8.00
Amazon S3 (frontend + seed)	~2GB storage, ~100GB transfer	\$0.025 CAD/GB storage	~\$3.50
Amazon CloudFront	~500GB data transfer	\$0.0135 CAD/GB (Canada)	~\$6.75
Amazon API Gateway	~5M REST API calls	\$4.25 CAD/million calls	~\$21.00
Amazon Bedrock (Claude 3 Sonnet)	~5,000 roadmap generations, ~3,000 tokens avg	\$0.005 CAD/1K input + \$0.025/1K output	~\$35.00
EC2 t3.micro (seed runner)	Negligible — runs once at launch	\$0.015 CAD/hr	~\$0.50
Amazon CloudWatch	~10GB log ingestion	\$0.76 CAD/GB over free tier	~\$3.80
Total			~\$83.05 CAD

5.3 Higher Cost Enhancements

To move to a professional production grade, the following enhancements are recommended:

- **AWS WAF (Web Application Firewall):** ~\$6.75 CAD/month base + rules. Protects against SQL injection and bots.
- **Amazon Route 53 (Managed DNS):** ~\$0.70 CAD/month for a custom domain zone.
- **Provisioned Concurrency:** ~\$15.00+ CAD/month to eliminate "Cold Starts" for a faster user experience.

6. Implementation Screenshots



7. Future Improvement

If development continues, PathMentor will evolve following these architectures:

1. AI Search Upgrade (RAG):

- Service: AWS Kendra or OpenSearch Serverless.
- Feature: Enable semantic search across millions of experiences to find "niche" differentiators that simple keyword filtering might miss.

2. Notification Engine:

- Service: Amazon Pinpoint.
- Feature: Send weekly "nudges" to users based on their personalized roadmap (e.g., "Check in: Did you start that networking strategy this week?").

3. Real-time Mentorship:

- Service: AWS AppSync (GraphQL).
- Feature: Real-time matching between mentees and experienced mentors for live Q&A.

4. User Authentication (Amazon Cognito):

- Implementing secure sign-in to allow users to save their roadmap and experiences.

8. Conclusion

PathMentor demonstrates the power of a modern, cloud-native architecture in solving real-world mentorship challenges. By utilizing AWS serverless services, the platform achieves high performance, security, and scalability while maintaining a near-zero cost for entry-level usage. The integration of Amazon Bedrock provides a unique value proposition that distinguishes PathMentor from traditional social platforms, offering actionable, context-aware guidance for the modern professional.